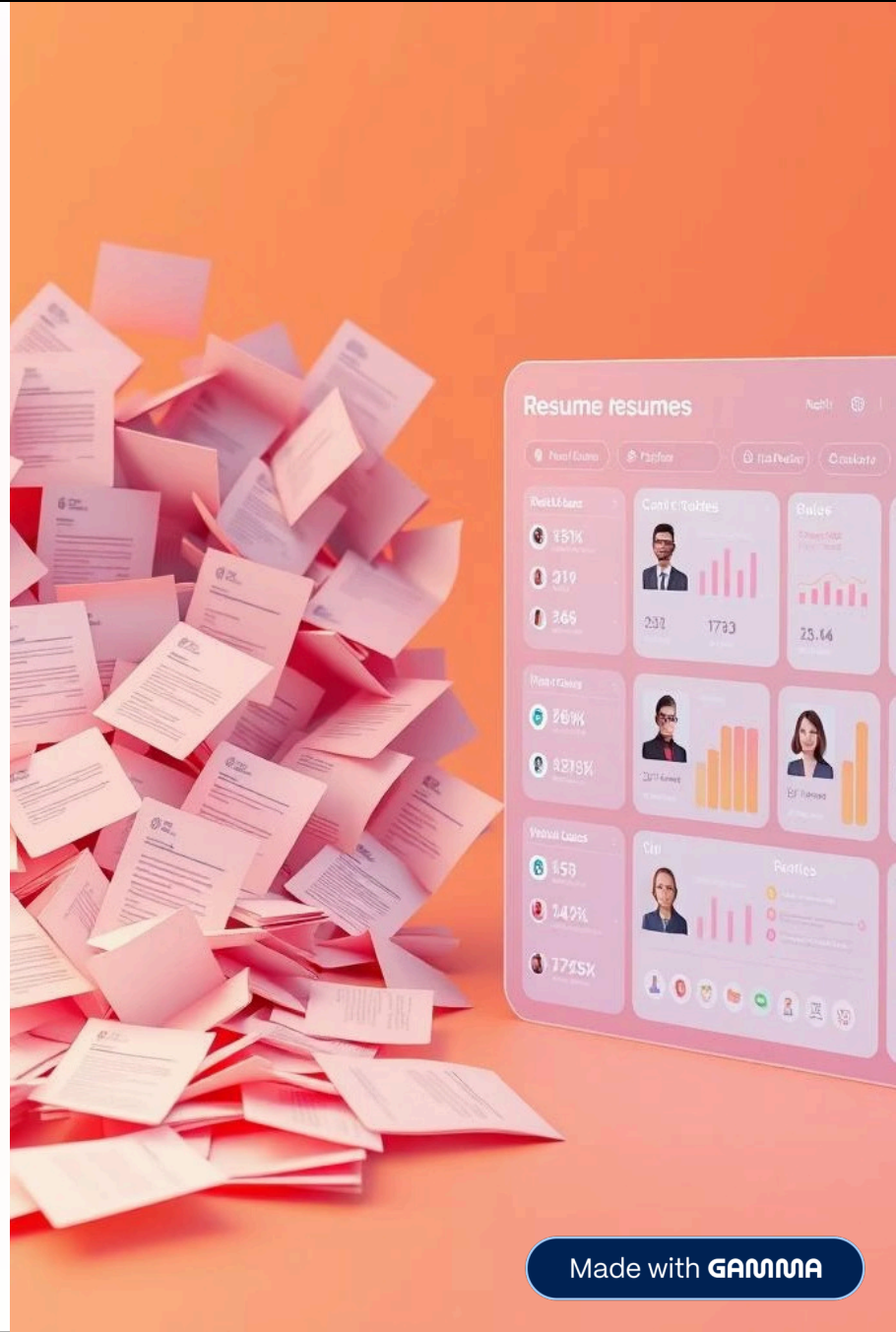
AI Resume Matching System

From Job Description to Ranked Candidates in Seconds

Su Yan

19/03/2026



Made with GAMMA

The Problem

200-500

Resumes per role

Volume overwhelms manual screening

23h

Average screening time

Per role (SHRM, 2023)

- Traditional ATS filters miss context and nuance
- Keyword matching fails on synonyms
- Qualified candidates rejected by mistake
- Hiring managers remain overwhelmed

Thesis: We need a smarter pipeline, not a faster keyword scanner.

Made with GAMMA

System Overview

1

Ingest

Searchable knowledge base

2

Retrieve

Relevant candidates

3

Evaluate

Multi-dimensional scores

4

Rank

Single comparable number

5

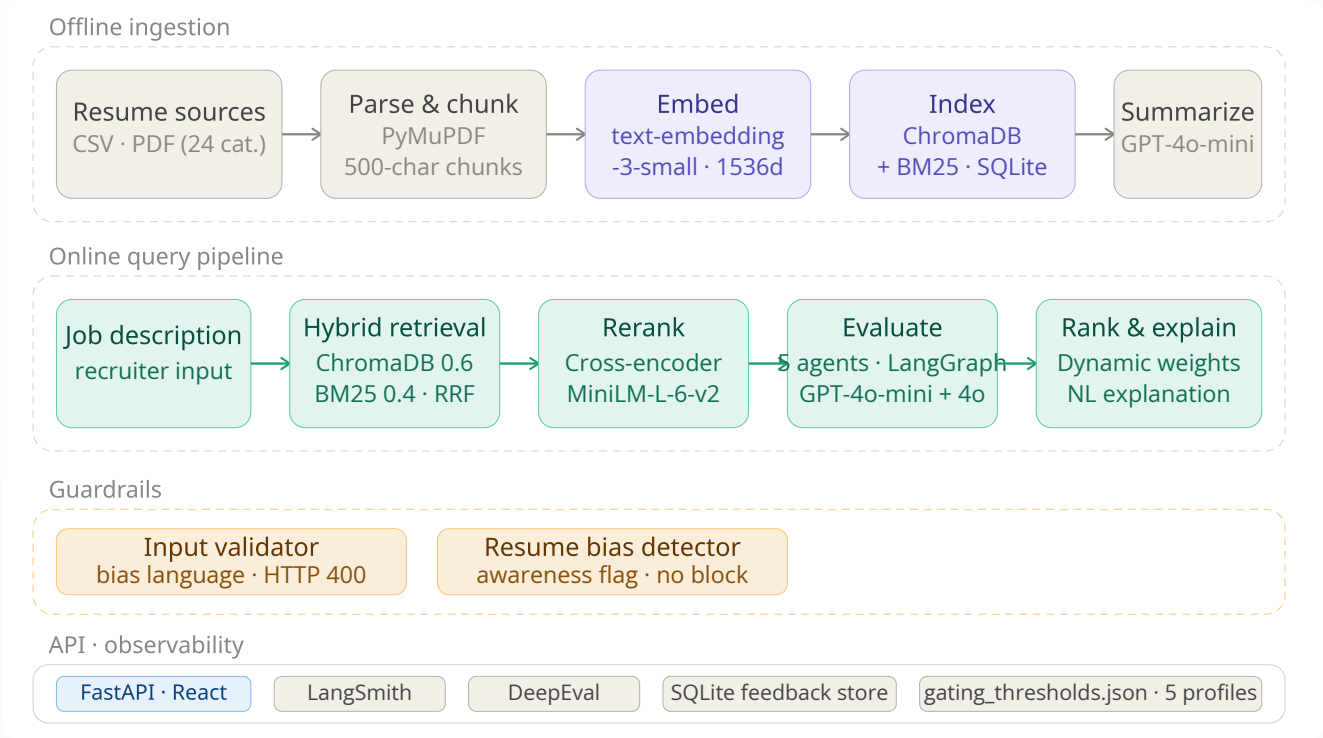
Explain

Why recommended

Each step solves a specific failure mode of traditional tools.

Made with GAMMA

Technical Architecture



Offline Phase

- Data ingestion and parsing
- Embedding & indexing with ChromaDB
- Resume summarization and caching

Online Phase

- Input validation and guardrails
- Hybrid retrieval (semantic + keyword)
- LangGraph pipeline with 6 nodes
- 5 specialist agents evaluation
- Score aggregation and explanation

Made with GAMMA

Design Decision #1: Hybrid Retrieval

Why Not Just Vector Search?

- Pure semantic search misses exact keywords
- Pure keyword search misses context
- Each fails predictably

Our Solution

Fuse both scores: semantic similarity + keyword match

Pure Semantic Search

Misses exact skills like "Kubernetes"

Pure Keyword Search

Misses synonyms like "container orchestration"

Hybrid Approach

Covers both failure modes

Design Principle: Use the right tool for each failure mode, then combine

Made with GAMMA

Design Decision #2: Multi-Agent Evaluation

One LLM cannot do everything well. Five specialist agents, each with a focused role.

Resume Parser

Heuristic-based, zero LLM cost, detects structure + bias signals

Skill Matching

Technical coverage with gpt-4o-mini

Experience Depth

Seniority signals with gpt-4o-mini

Technical Depth

Project complexity with gpt-4o-mini

Culture Fit

Soft skills, needs gpt-4o reasoning

Design Principle: Decompose complex judgment into narrow, verifiable sub-tasks

Made with GAMMA

Design Decision #3: Token Optimization

Resumes Are Long. LLM Budgets Are Real.

The Challenge

Full resume = 8,000 tokens × 5 candidates × 4 scoring agents

Bad Solution

Truncate text — loses certifications and projects

01

GPT-4o-mini pre-summarizes

Structured digest preserving key sections

02

Persistent disk cache

LLM called once per resume, zero repeat cost

Design Principle: Compress information, not just text

Made with GAMMA

Design Decision #4: Guardrails

The System Needs to Know What NOT to Do

1

Input Guardrail

Blocks biased queries before pipeline entry

"male only developer" → HTTP 400 with clear UI message

2

Resume Bias Detector

Flags demographic disclosures

Shown as recruiter awareness warning

Design Principle: Safety is a feature, not an afterthought

Made with GAMMA

Design Decision #5: Dynamic Weighting & Gating

One Size Does Not Fit All

Different job categories (e.g., Technical, Management, Analytics) require distinct evaluation priorities to ensure the best fit.

Dynamic Weighting

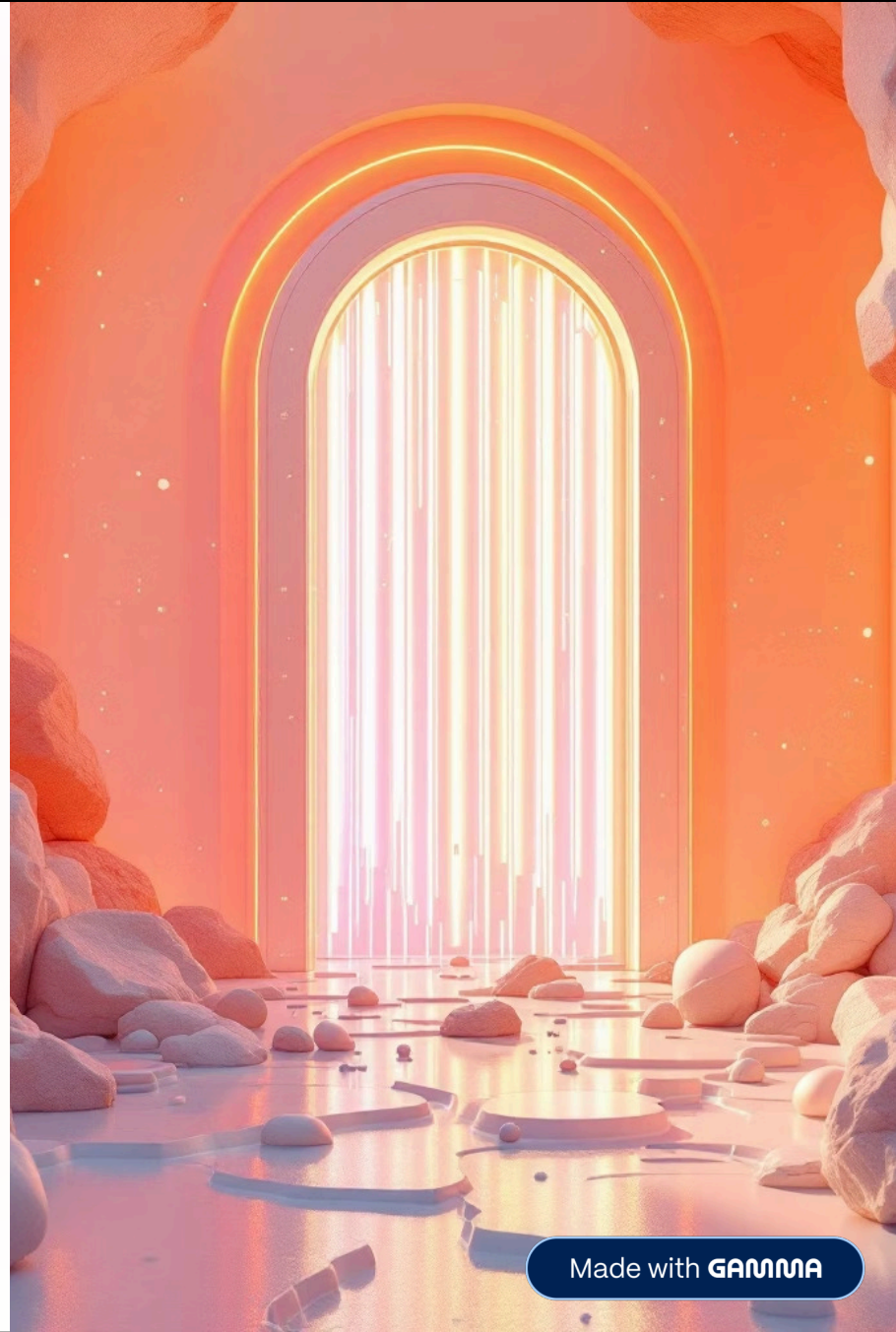
The system automatically selects the weight profile based on the job category to emphasize the most relevant competencies.

Job Category	Skill	Tech Depth	Experience	Culture Fit
Technical	0.40	0.20	0.25	0.15
Management	0.20	0.15	0.35	0.30
Analytics	0.35	0.30	0.20	0.15

Post-Agent Gating

If a candidate scores critically low in a "must-have" skill, the gating mechanism prevents high "culture fit" scores from inflating their rank, effectively avoiding false positives. For example, if a role explicitly requires Python, a candidate lacking this skill will be blocked regardless of their other strengths.

Design Principle: Context matters. Adapt scoring to the job, not the candidate to the scoring.



Made with GAMMA

Observability & Quality

Built to Be Measured, Not Just to Work

LangSmith Integration

- Every pipeline node traced automatically
- Per-node token counts and latency visible
- Full run history for debugging

DeepEval Integration

- AnswerRelevancyMetric scores output quality
- GEval provides LLM-as-judge rubric
- Test results saved to JSON

Known latency tradeoff: 4 agents × 5 candidates = 20 sequential LLM calls. Next: parallelize agents per candidate.

Made with GAMMA

Live Demo Preview

What You're About to See

Biased Query Blocked

Guardrail shows clear, non-scary UI message

Valid Query Runs

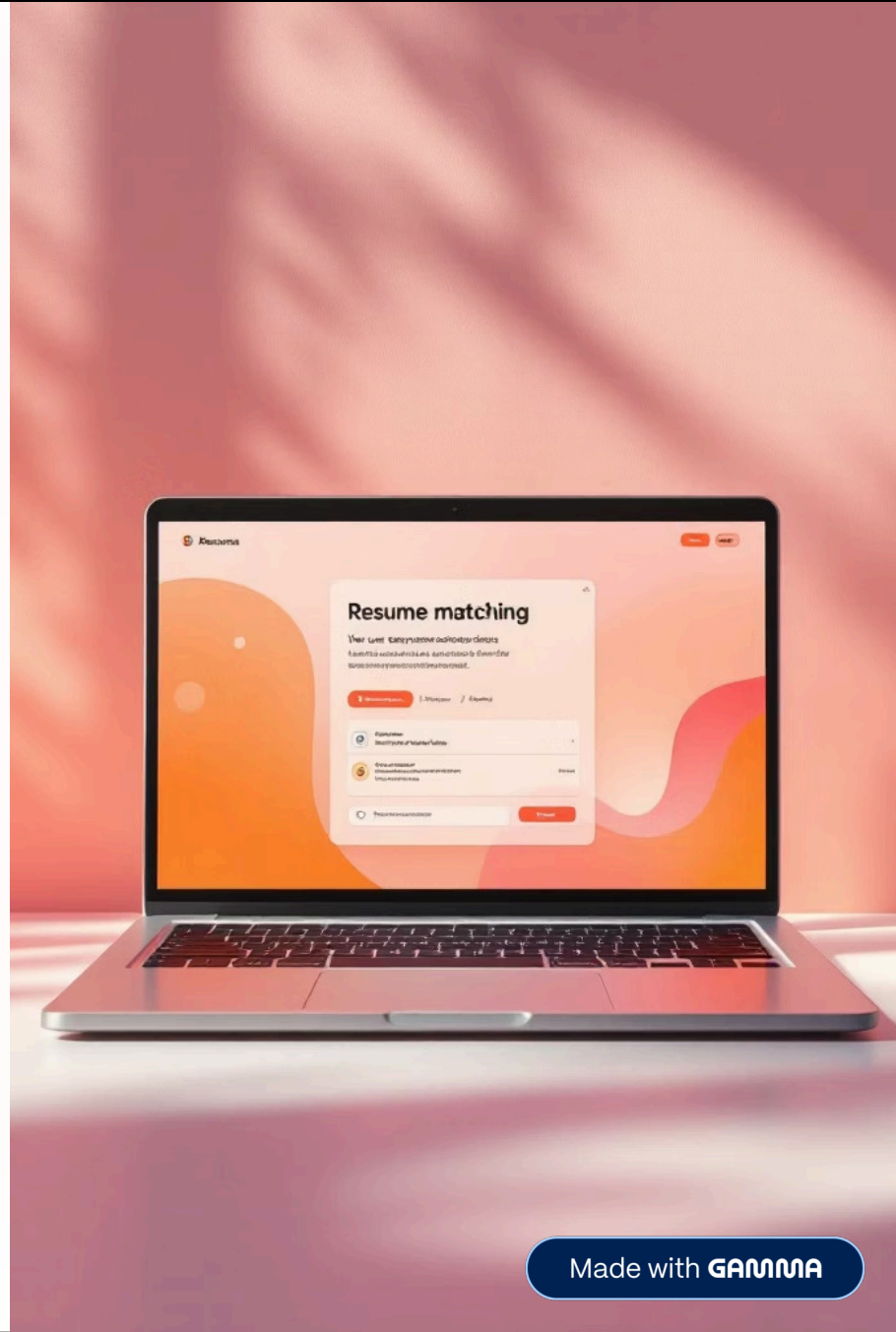
Pipeline executes, ranked results appear with scores

Click Candidate

Score breakdown per dimension + bias flag warning

Analytics Page

Feedback stats dashboard visible



Made with GAMMA

Key Takeaways

🔍 Hybrid Beats Pure

Combine strategies to cover blind spots

🤖 Decompose Judgment

Specialist agents outperform generalist prompts

🛡️ Design for Trust

Guardrails, explainability, observability from day one

> "The hardest part wasn't the AI. It was designing a system a recruiter can actually trust."

Made with GAMMA

Project Repository

GitHub Repository

https://github.com/shuyan01/capstone

Includes:

- Full source code
- RAG pipeline implementation
- Evaluation scripts
- Documentation

Made with GAMMA

Thank you for your attention!

Made with GAMMA